```python
# python3 -m pip install tabulate
from tabulate import tabulate

# Participating students:
# Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran

# Constructs a Normalized Linear Approximation Table (NLAT) for a given S-box and bit length l.
def construct_nlat(sbox, l):
    num_elements = 2**l
    midpoint = 2**(l - 1) # Normalization factor: 4 for l=3
    table = []

    for a in range(num_elements):
        row = []
        for b in range(num_elements):
            matches = 0
            for x in range(num_elements):
                # Calculate bitwise dot product parity for input and output
                input_parity = bin(x & a).count('1') % 2
                output_parity = bin(sbox[x] & b).count('1') % 2

                if input_parity == output_parity:
                    matches += 1

            # Apply normalization: N_L(a, b) - 4
            row.append(matches - midpoint)
        table.append(row)
    return table

# Calculates the subkey counts for a 3-bit subkey, given:
#   - A set of plaintext/ciphertext pairs (stored as seperate lists)
#   - An inverse of the sbox function
# Returns a dictionary with all posssible subkey guesses and their corresponding counters
def calc_subkey_counts(plaintext: list, ciphertext: list, inverse_sbox: dict):

    # Storages for guesses and counts
    subkey_counts = {0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0}

    for subkey in range(0, 8):
        # For each subkey, check each plaintext/cipertext pair
        for i in range(0, len(plaintext)):
            # Get first 3 bits of cipertext, and do bitwise
            # addition with subkey guess to recover first 3 bits of v
            v = (ciphertext[i] >> 3) ^ subkey
            # Apply inverse S-box to recover first 3 bits of u
            u = inverse_sbox[v]
            plain = plaintext[i]
            # Get needed plaintext bits
            p1 = (plain >> 5) & 1
            p2 = (plain >> 4) & 1
            p4 = (plain >> 2) & 1
            p5 = (plain >> 1) & 1
            # Get h1
            h1 = (u >> 2) & 1
            # Compute bitwise sum
            z = p1 ^ p2 ^ p4 ^ p5 ^ h1
            # If sum is 0, increment counter
            if z == 0:
                subkey_counts[subkey] += 1
    return subkey_counts

def main():
    # S-Box definition:
    # Input:  0, 1, 2, 3, 4, 5, 6, 7
    # Output: 6, 5, 1, 0, 3, 2, 7, 4
    sbox = {0: 6, 1: 5, 2: 1, 3: 0, 4: 3, 5: 2, 6: 7, 7: 4}
    # Inverse S-Box definition:
    # Input:  0, 1, 2, 3, 4, 5, 6, 7
    # Output: 3, 2, 5, 4, 7, 1, 0, 6
    inverse_sbox = {0: 3, 1: 2, 2: 5, 3: 4, 4: 7, 5: 1, 6: 0, 7: 6}

    # Set S-box bit length
    l = 3

    # Create the Normalized Linear Approximation Table (NLAT)
    nlat_table = construct_nlat(sbox, l)

    # Format the table for output using tabulate
    headers = ["InputSums/OutputSums"] + [f"{i}" for i in range(2**l)]
    table_string = tabulate(nlat_table, headers=headers, showindex="always", tablefmt="grid")

    # Find total bias of trail
    n = 3 # Number of S-boxes in the trail
    total_bias = (2 ** (n - 1)) * (( nlat_table[6][4] ) / (2 ** l)) ** 3

    # Set plaintext and ciphertext sets
    # Binary strings are represented in decimal form for simplicity
    plaintext = [39, 7, 12, 24, 8, 26]
    ciphertext = [36, 50, 57, 29, 13, 41]


    # Initialize all associated subkey guess counters to 0
    # Note binary guesses are represented in decimal form for simplicity
    subkey_counts = calc_subkey_counts(plaintext, ciphertext, inverse_sbox)


    # Used for formatting output
    decimal_binary = {0: "000", 1: "001", 2: "010", 3: "011", 4: "100", 5: "101", 6: "110", 7: "111"}

    # Size of our plaintext/cipertext set
    tau = len(plaintext)

    # Normalize counts by distance from tau/2
    normalized_counts = {subkey: abs(subkey_counts[subkey] - tau/2) for subkey in subkey_counts}

    # Save the NLAT to the output file
    with open("Project3.txt", "w") as f:
        f.write(f"Project 3: Cryptography MATH 4175\n")
        f.write(f"Participating Students:\n")
        f.write(f"Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran\n\n")
        f.write("Part 1: Normalized Linear Approximation Table (NL(a,b) - 4)\n")
        f.write("Rows: Input Sums, Columns: Output Sums\n\n")
        f.write(table_string)
        f.write(f"Based on the table, we chose a path starting with the input sum of 110 and an output sum of 100 . A visualization of this path is attached.\n")
        f.write(f"\n\nTotal Bias: {total_bias}\n\n")
        f.write(f"Subkey Guesses and Corresponding Counter Values:\n")
        for subkey in range(0, 8):
            f.write(f"  {decimal_binary[subkey]}: {subkey_counts[subkey]}\n")
        f.write(f"\nSubkey Guesses and Normalized distance from tau/2:\n")
        for subkey in range(0, 8):
            f.write(f"  {decimal_binary[subkey]}: {normalized_counts[subkey]}\n")

if __name__ == "__main__":
```

```
main()
```